

Docker Compose

Nginx로드밸런싱과 다중 환경 분기

학습 목표

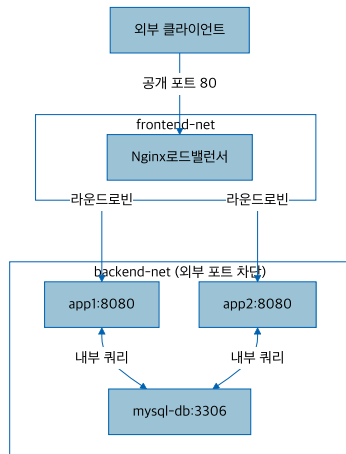
- 외부 요청을 수신하는 Nginx, 비즈니스 로직을 처리하는 애플리케이션, 데이터를 보관하는 데이터베이스를 3-Layer로 분리합니다
- `upstream` 라운드로빈으로 두 애플리케이션 인스턴스에 트래픽을 분산하고 원본 클라이언트 정보를 프록시 헤더로 보존합니다
- 백엔드와 데이터베이스의 호스트 포트 노출을 제거해 오직 Nginx만 외부에 공개하는 심층 방어 구조를 구성합니다
- `compose.override.yaml` / `compose.prod.yaml` 다중 파일 오버라이드로 로컬과 프로덕션 환경을 하나의 베이스 명세에서 분기합니다

2-Tier 구성에서는 애플리케이션이 한 대뿐이라 장애나 트래픽 증가에 취약했습니다. 본 차시는 애플리케이션을 이중화하고 그 앞에 **Nginx로드밸런서**를 배치해, 네트워크 격리와 트래픽 분산을 동시에 갖춘 구조로 확장합니다.

핵심 개념 및 구조

3-Layer 아키텍처: 계층별 사설망 분리

외부 요청은 Nginx만 거치고, 애플리케이션과 데이터베이스는 각각 다른 사설망에 격리됩니다.



🏠 단일 진입점

외부 포트는 Nginx의 **80** 하나뿐입니다

🏢 무상태 스케일아웃

app1, **app2**는 동일 이미지의 복제본입니다

🏡 데이터 완전 격리

DB는 **backend-net** 안에서만 조회됩니다

계층별 격리 비교

계층	서비스	소속 네트워크	외부 포트
Presentation	nginx	frontend-net	80:80 공개
Application	app1, app2	frontend-net, backend-net	없음 (내부 전용)
Data	db	backend-net	없음 (완전 차단)

애플리케이션 계층이 두 사설망에 동시에 소속되는 것이 핵심입니다. 프론트엔드 망으로는 Nginx의 요청을 받고, 백엔드 망으로는 DB를 조회하는 중계 역할을 합니다.

다중 환경 분기 전략: local vs prod

환경	적용 파일 조합	포트 노출
로컬 개발	<code>.env.local</code> + <code>compose.yaml</code> + <code>compose.override.yaml</code>	<code>8080</code> , <code>3306</code> 직접 개방
프로덕션	<code>.env.prod</code> + <code>compose.yaml</code> + <code>compose.prod.yaml</code>	<code>80</code> (Nginx) 만 공개

📁 단일 베이스, 다중 오버라이드

공통 골격은 `compose.yaml` 하나에만 선언합니다

🔗 차이만 오버라이드

포트 개방 여부와 Nginx 포함 여부만 환경별로 달라집니다

실습 단계

실습 로드맵: 3-Layer 구성부터 환경 분기까지

구간	주제	이 구간에서 직접 확인하는 것
1	Nginx upstream 설정	라운드로빈 대상 서버 그룹 정의
2	프록시 헤더 검증	원본 클라이언트 IP·호스트명 보존 규칙
3	3-Layer compose.yaml 선언	Nginx·app1·app2·db 서비스 명세
4	서비스 구동과 검증	포트 격리와 라운드로빈 동작
5	.env 환경 분기	local·prod 자격증명과 메시지 분리
6	오버라이드 병합 구동	환경별 포트 차이와 정리 절차

앞 구간이 **3-Layer 구조 자체**라면 뒤 구간은 **같은 구조를 환경별로 다르게 배포**하는 절차입니다.

Nginx upstream 라운드로빈 설정

`app1`, `app2` 두 대로 트래픽을 분산할 업스트림 그룹과 프록시 규칙을 정의합니다.

```
events { worker_connections 1024; }
http {
    upstream backend_servers {
        server app1:8080;
        server app2:8080;
    }
    server {
        listen 80;
        location / {
            proxy_pass http://backend_servers;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For
                $proxy_add_x_forwarded_for;
        }
    }
}
```

지시어	기능
<code>upstream</code> <code>backend_servers</code>	<code>app1:8080</code> , <code>app2:8080</code> 을 단일 논리 그룹으로 결합
<code>proxy_pass</code>	인바운드 요청을 업스트림 그룹으로 전달

기본 알고리즘은 **라운드로빈**입니다. 별도 지시어 없이도
두 서버로 순서대로 분산됩니다.

클라이언트 원본 헤더 보존

Nginx가 요청을 중계할 때 클라이언트의 실제 IP와 호스트명이 소실되지 않도록 주입하는 헤더를 점검합니다.

```
cat nginx/nginx.conf | grep proxy_set_header
```

헤더	전달 데이터	필요성
<code>Host \$host</code>	클라이언트가 요청한 실제 도메인	리다이렉트 및 가상 호스트 처리
<code>X-Real-IP \$remote_addr</code>	직접 연결된 클라이언트 IP	단일 원격 접속지 추적
<code>X-Forwarded-For</code>	경유한 모든 프록시 IP 체인	감사 로그와 원본 IP 복원

프록시 헤더를 주입하지 않으면 백엔드 애플리케이션은 클라이언트의 실제 IP 대신 Nginx 컨테이너의 사설 IP만 `remote_addr`로 수신합니다.

3-Layer compose.yaml: Nginx와 이중화 애플리케이션

```
services:
  nginx:
    image: nginx:alpine
    ports: ["80:80"]
    depends_on: [app1, app2]
    networks: [frontend-net]
  app1:
    build: .
    environment: { APP_MESSAGE: "Instance #1 (app1)" }
    networks: [frontend-net, backend-net]
  app2:
    build: .
    environment: { APP_MESSAGE: "Instance #2 (app2)" }
    networks: [frontend-net, backend-net]
```

서비스	연결 네트워크	설명
nginx	frontend-net	유일한 외부 공개 엔드포인트
app1, app2	frontend-net, backend-net	두 사설망을 모두 연결하는 중계 인스턴스

3-Layer compose.yaml: 격리된 데이터베이스

`db` 서비스는 `backend-net`에만 소속되며 호스트 포트 매핑이 없습니다.

```
db:
  image: mysql:8.0
  environment: { MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}, MYSQL_DATABASE: ${DB_NAME} }
  volumes: ["db-data:/var/lib/mysql"]
  networks: [backend-net]
volumes: { db-data: }
networks: { frontend-net: , backend-net: }
```

설정	보안 이점
<code>networks: [backend-net]</code>	Nginx와 외부 인터넷에서 DB에 직접 접속 불가
<code>ports</code> 항목 없음	호스트 <code>3306</code> 포트를 열지 않아 무차별 침입 방지

`docker compose config`로 `nginx`만 `published: "80"`이고 나머지 서비스는 `ports` 항목이 없는지 사전 검증합니다.

3-Layer 서비스 구동과 DB 준비 확인

4개 서비스를 백그라운드로 일괄 구동하고, MySQL 준비 완료 여부를 확인한 뒤 백엔드 인스턴스를 재구동합니다.

```
docker compose up -d
docker compose logs db | grep "ready for connections"
docker compose restart app1 app2
docker compose ps
```

명령	기능
<code>docker compose up -d</code>	4개 컨테이너와 2개 사설망 일괄 생성
<code>docker compose logs db</code>	3306 포트 수신 대기 상태 확인
<code>docker compose restart app1 app2</code>	커넥션 풀을 확실하게 초기화

라운드로빈 트래픽 분산 검증

호스트의 80 포트로 요청을 반복 전송해 두 백엔드 인스턴스가 번갈아 응답하는지 확인합니다.

```
curl -s http://localhost/  
curl -s http://localhost/  
curl -s http://localhost/  
curl -s http://localhost/  
  
# Nginx 접근 로그 실시간 스트리밍  
docker compose logs -f nginx
```

호출 순번	응답 인스턴스	라우팅 방식
1회차 · 3회차	app1	upstream 목록의 첫 번째 서버
2회차 · 4회차	app2	upstream 목록의 두 번째 서버

클라이언트는 `nginx` 한 곳에만 접속하며, 백엔드 두 대로의 분산은 `upstream` 내부에서만 일어납니다.

환경별 설정 분리: `.env.local` vs `.env.prod`

동일한 명세 파일을 그대로 두고 값만 파일로 분리해 로컬과 운영 환경을 구분합니다.

변수	<code>.env.local</code>	<code>.env.prod</code>
<code>DB_PASSWORD</code>	<code>localpass</code>	운영용 강력한 비밀번호
<code>APP_MESSAGE</code>	로컬 개발 모드 안내 문구	프록시 경우 운영 모드 안내 문구
공통 변수	<code>DB_NAME</code> , <code>DB_USER</code> , HikariCP 지연 연결 옵션	

두 파일 모두 자격증명을 담으므로 `.env.local`, `.env.prod`는 형상 관리 대상에서 제외합니다.

공통 명세와 로컬 오버라이드

```
# compose.yaml (공통)
services:
  app:
    build: .
    environment:
      APP_MESSAGE: ${APP_MESSAGE}
      SPRING_DATASOURCE_URL: "jdbc:mysql://db:3306/${DB_NAME}"
    depends_on: [db]
    networks: [app-net]
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
    networks: [app-net]
volumes: { db-data: }
networks: { app-net: }
```

```
# compose.override.yaml (local 전용)
services:
  app:
    ports:
      - "8080:8080"
  db:
    ports:
      - "3306:3306"
```

`compose.override.yaml`은 별도 플래그 없이 **자동 병합**되어 로컬 디버깅용 포트를 개방합니다.

운영 전용 확장: `compose.prod.yaml`

포트 대신 Nginx 프록시를 전면에 배치하고 재시작 정책을 강화합니다.

```
services:
  app: { restart: always }
  db: { restart: always }
  nginx:
    image: nginx:alpine
    restart: always
    ports: ["80:80"]
    volumes: ["/nginx/nginx.conf:/etc/nginx/nginx.conf:ro"]
    depends_on: [app]
    networks: [app-net]
```

오버라이드 속성	효과
<code>restart: always</code>	컨테이너 무중단 자동 재시작
<code>nginx</code> 서비스 추가	<code>80</code> 포트만 공개, <code>app</code> / <code>db</code> 는 비공개 유지

local vs prod 병합 구동과 포트 차이 검증

```
docker compose down
```

```
# 로컬: 베이스 + override 자동 병합
```

```
docker compose --env-file .env.local up -d
```

```
curl http://localhost:8080/
```

```
docker compose down
```

```
# 운영: 베이스 + prod 확장 명시 병합
```

```
docker compose --env-file .env.prod \  
-f compose.yaml -f compose.prod.yaml up -d
```

```
curl http://localhost/
```

모드	활성 포트	응답 메시지
로컬 개발	8080, 3306	로컬 개발 모드 안내
프로덕션	80만 공개	프록시 경유 운영 모드 안내

같은 `compose.yaml`을 두고 어떤 파일을 겹쳐 쓰느냐만으로 배포 형태가 달라집니다.

전체 정리: 컨테이너·네트워크·볼륨 일괄 삭제

```
docker compose -f compose.yaml -f compose.prod.yaml down -v
docker ps -a
docker volume ls
```

명령	삭제 대상
<code>down -v</code>	컨테이너, <code>app-net</code> , 명명 볼륨 <code>db-data</code> 전체
<code>docker ps -a</code> / <code>docker volume ls</code>	잔여 리소스 없음을 최종 확인

운영 환경에서는 `-v` 사용을 원칙적으로 금지하며, 실습 종료 시점의 정리 목적으로만 사용합니다.

한 장 요약

영역	핵심 요약	비고
3-Layer 격리	웹(Presentation) - 앱(Application) - DB(Data) 망 분리	DB 외부 포트 노출 차단
부하 분산	Nginx 리버스 프록시 기반 다중 앱 Round Robin 분산	단일 엔드포인트(80) 서비스 제공
환경 분기	<code>.env.local</code> vs <code>.env.prod</code> , <code>compose.prod.yaml</code> 오버라이드	단일 베이스 명세 다중 환경 확장
서비스 연속성	특정 애플리케이션 인스턴스 장애 시 가용성 유지	인프라 이중화 및 SPOF 최소화

3-Layer 아키텍처와 Compose 환경 오버라이드를 결합하면 개발 환경의 유연성과 운영 환경의 보안 격리를 동일한 코드베이스로 제어할 수 있습니다.

Appendix

Nginx 설정 문법 기초: 컨텍스트와 지시어

Nginx 설정은 중괄호로 중첩된 컨텍스트 계층 구조이며, 하위 컨텍스트는 상위 설정을 상속합니다.

```
events { ... }      # 전역
http {              # HTTP 전역
    upstream NAME { ... }
    server {        # 가상 서버
        location PATH { ... }
    }
}
```

지시어	컨텍스트	역할
<code>listen</code>	<code>server</code>	수신 포트 지정
<code>location</code>	<code>server</code>	경로별 처리 분기
<code>proxy_pass</code>	<code>location</code>	업스트림으로 전달
<code>proxy_set_header</code>	<code>location</code>	전달 헤더 치환

지시어는 **이름 값;** 형태로 세미콜론을 붙이며, 블록 지시어는 세미콜론 대신 **{ }**로 하위 컨텍스트를 엽니다.

로드밸런싱 확장과 단일 장애점

⚙️ 가중치 분산

서버 성능이 다르면 `server app1:8080 weight=2;`로
분산 비율을 조정합니다

⚠️ 단일 장애점

Nginx 한 대만 운영하면 그 컨테이너가 곧 전체
서비스의 장애점이 됩니다

Nginx 자체의 이중화는 본 실습 범위 밖의 확장 시나리오이며, AWS 관리형 로드밸런서를 다루는 클라우드 배포 실습에서 이어집니다.

HTTPS 처리: SSL/TLS 종단과 Let's Encrypt

✓ SSL/TLS 종단

클라이언트-Nginx 구간만 암호화하고, Nginx-`app` 구간은 `frontend-net` 내부에서 평문 HTTP로 통신합니다

📖 Let's Encrypt · Certbot

무료 CA인 `Let's Encrypt`가 `ACME` 프로토콜의 `HTTP-01 challenge`로 도메인 소유권을 검증해 인증서를 발급하고, `Certbot`이 발급과 자동 갱신을 대행합니다

본 실습은 도메인 없이 `localhost`로 진행되어 소유권 검증이 불가능하므로 `compose.prod.yaml`도 80번(HTTP)만 개방합니다. 실제 배포(EC2 등)에서 도메인을 연결한 뒤에는 `certbot --nginx`로 443번 포트와 HTTPS 리다이렉트를 추가 구성합니다.

서브도메인 라우팅과 인증서 구성

🔑 `server_name` 가상 호스트

하나의 IP·포트로 유입된 요청을 `Host` 헤더 기준 여러 `server { }` 블록으로 분기해 서브도메인별로 다른 백엔드에 `proxy_pass` 합니다

📜 SAN · 와일드카드 인증서

인증서는 도메인 단위 발급이 원칙이라 서브도메인이 여러 개면 `-d` 다중 지정으로 묶거나, `*.example.com` 와일드카드 인증서를 `DNS-01 challenge` 로 발급받습니다

Nginx는 하나의 공인 IP·포트로도 다수의 서비스를 도메인 기준으로 분리해 운영할 수 있게 하는 트래픽 진입점 역할을 합니다.

핵심 기술 면접 질문

3-Tier 네트워크 격리의 보안상 이점

3-Tier 아키텍처에서 백엔드와 데이터베이스의 포트를 외부에 열지 않고 Nginx만 포트를 여는 보안상 이점은 무엇인가요?

X-Forwarded-For 헤더의 필요성

Nginx 리버스 프록시 환경에서 `proxy_set_header X-Forwarded-For`를 설정해야 하는 이유는 무엇인가요?

다중 파일 오버라이드 원리

Docker Compose에서 개발 환경(local)과 운영 환경(prod)을 다중 파일 오버라이드로 분기하는 원리는 무엇인가요?

로드밸런서 단일 장애점(SPOF) 위험

대규모 트래픽 환경에서 Nginx로드밸런서를 이중화(HA)하지 않았을 때 발생할 수 있는 아키텍처적 위험(SPOF)은 무엇인가요?

핵심 기술 면접 질문

Let's Encrypt 인증서 발급 제약

도메인 없이 `localhost`나 공인 IP만으로 `Let's Encrypt` 인증서를 발급받을 수 없는 이유는 무엇인가요?

서브도메인 다중 서비스의 Nginx 인증서 구성

서브도메인이 여러 개인 서비스에서 Nginx와 인증서를 어떻게 구성해야 하나요?

모범 답변

3-Tier 네트워크 격리의 보안상 이점

심층 방어(Defense in Depth) 원칙을 적용하기 위함입니다. 외부 인터넷과 직접 통신하는 Nginx만 공개 포트(80/443)를 열고, 애플리케이션과 DB는 포트 매핑을 제거해 내부 가상 사설망(`frontend-net`, `backend-net`)에 격리합니다. 이로 인해 외부에서 DB 포트(3306)로 직접 무차별 대입 공격을 시도하거나 알려진 DB 취약점을 노리는 공격 경로를 원천 차단합니다.

X-Forwarded-For 헤더의 필요성

Nginx가 리버스 프록시로 중계하면 백엔드 스프링부트가 수신하는 원격 IP(`remote_addr`)는 실제 사용자가 아니라 Nginx 컨테이너의 내부 사설 IP가 됩니다. 따라서 접속 통계 수집, 사용자 IP 기반 인증, 부정 접속 방지(FDS) 등을 정상 수행하려면 Nginx가 클라이언트 원본 IP를 `X-Forwarded-For` 헤더에 담아 백엔드로 전달해야 합니다.

다중 파일 오버라이드 원리

`compose.yaml`에 공통 설정(서비스 구조, 베이스 이미지, 네트워크)을 선언하고, `compose.override.yaml`(로컬)이나 `compose.prod.yaml`(운영)을 덧씌워 환경별 차이점(포트 개방 여부, 환경변수, Nginx 프록시 포함 여부)만 선택적으로 병합(Merge)하는 방식입니다. 이를 통해 중복 코드를 없애고 단일 원본 기반으로 환경별 배포를 안정적으로 관리합니다.

모범 답변

로드밸런서 단일 장애점(SPOF) 위험

Nginx가 단 1대만 존재할 경우, 트래픽을 처리하는 Nginx 컨테이너나 호스트 서버가 다운되면 뒤에 있는 백엔드 인스턴스가 아무리 정상 작동하더라도 전체 서비스가 중단됩니다. 이를 방지하기 위해 실무 클라우드 환경에서는 AWS ALB 같은 관리형 로드밸런서를 전면에 두거나, Keepalived/VRRP를 이용해 Nginx를 액티브-스탠바이 또는 다중화로 구성합니다.

Let's Encrypt 인증서 발급 제약

Let's Encrypt는 ACME 프로토콜의 HTTP-01 challenge로 도메인 소유권을 검증하는데, 이는 요청받은 도메인 이름(FQDN)의 DNS 레코드가 실제로 해당 서버를 가리키는지 확인하는 절차입니다. localhost나 공인 IP 자체는 소유권을 증명할 도메인 개념이 없어 검증 대상이 성립하지 않으므로 발급이 거부됩니다. HTTPS를 적용하려면 도메인을 구매해 A 레코드를 연결한 뒤 인증서를 발급받아야 합니다.

서브도메인 다중 서비스의 Nginx·인증서 구성

Nginx 측에서는 server_name 지시어로 구분된 여러 server { } 블록을 두어 Host 헤더 기준으로 서브도메인별 백엔드에 proxy_pass합니다. 인증서 측에서는 서브도메인 목록을 -d 옵션으로 나열해 하나의 SAN 인증서에 묶거나, 서브도메인이 유동적으로 늘어나는 경우 *.example.com 와일드카드 인증서를 DNS-01 challenge 방식으로 발급받아 대응합니다.

깊이 알아보기

Nginx HTTP로드밸런싱 HTTP Load Balancing

업스트림 분배 알고리즘 및 가중치 설정

<https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/>

Nginx 리버스 프록시 Reverse Proxy

헤더 전달 및 버퍼링 공식 가이드

<https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/>

Docker Compose 다중 파일 분기 Multiple Compose files

환경별 오버라이드 병합 규칙 원문

<https://docs.docker.com/compose/how-tos/multiple-compose-files/>

Compose 프로덕션 가이드 Compose in production

운영 배포 시 권장 아키텍처 및 보안

<https://docs.docker.com/compose/how-tos/production/>

깊이 알아보기

Let's Encrypt 작동 원리 How It Works

ACME 프로토콜 기반 도메인 유효성 검증

<https://letsencrypt.org/howitworks/>

Certbot Nginx 설치 및 갱신 Certbot Instructions

자동 인증서 발급 및 주기적 갱신 설정

<https://certbot.eff.org/instructions>



- EOF -